12

State Management in React

In the previous chapters, we explored the concept of state in React and mastered the basics of working with it using the `useState` hook. Now it's time to delve deeper into the **global state management** of applications. In this chapter, we will focus on the global state: we'll define what it is, its key advantages, and the strategies for its effective management.

This chapter will cover the following topics:

- What is global state?
- React Context API and useReducer
- Redux
- Mobx

What is global state?

In developing React applications, one of the key aspects that requires special attention is **state management**. We are already familiar with the `useState` hook, which allows us to create

and manage state within a component. This type of state is often referred to as **local**, and it is very effective within a single component and very simple and easy to use.

For a clearer illustration, consider an example with a small form component, where we have two **input** elements and have created two states for each **input**:

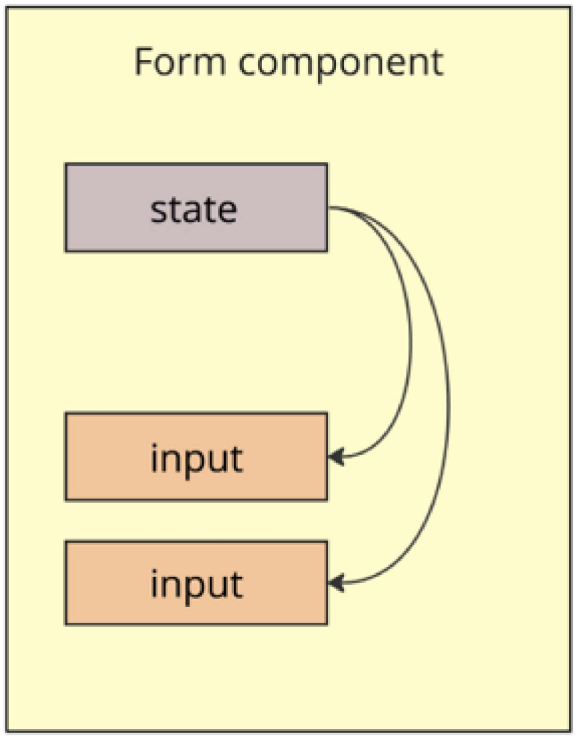

Figure 12.1: Form component with local state

In this example, everything is simple: the user enters something into the `input`, which triggers an `onChange` event, where we usually change our `state`, causing a full re-render of the form, and then we see the result of the input on the screen.

However, as the complexity and size of your application increase, there will inevitably be a need for a more scalable and flexible approach to state management. Let's further consider our example and imagine that after entering information into the form, we need to make a request to the server for user authorization and obtain a **session key**. Then, with this key, we need to request user data: name, surname, and avatar.

Here, we immediately encounter difficulties: where do we store the session key and user data? Perhaps we can retrieve the data right inside the form and then pass it up to the parent component, as it is more global and responsible. Alright, let's illustrate this and take a look:

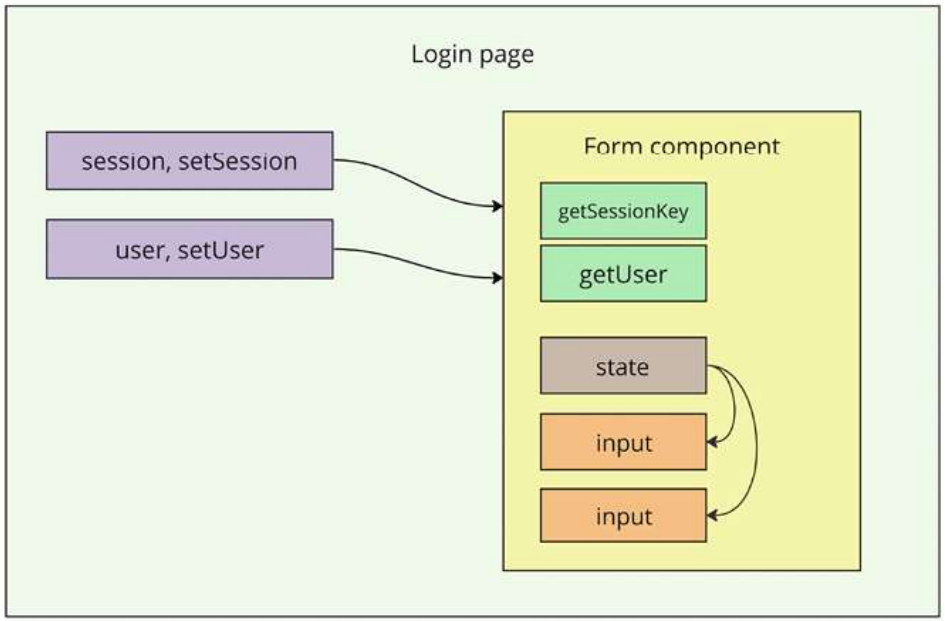

Figure 12.2: Login page with form component

So, now we have a login page, where we have local states for **session** and **user** objects. Using props, we can pass functions like `onSessionChange` and `onUserChange` to the form component, which ultimately allows us to transfer data from the form to the login page. Also, in the form, we now have the functions `getSessionKey` and `getUser`. These methods interact with the server, and upon successful response, they don't store data locally but call the aforementioned `onSessionChange` and `onUserChange`.

One might think that the data storage problem is solved, but likely after user authorization and obtaining their data, we need to redirect the user to some homepage of our application. We could repeat our trick of lifting the data higher once again, but before doing that, let's think ahead and imagine that obtaining user data is probably not just the job of the authorization form, and such a function might be needed on other pages.

Ultimately, we come to understand that in addition to the data itself, we also need to keep the logic for working with the data higher up in the component tree:

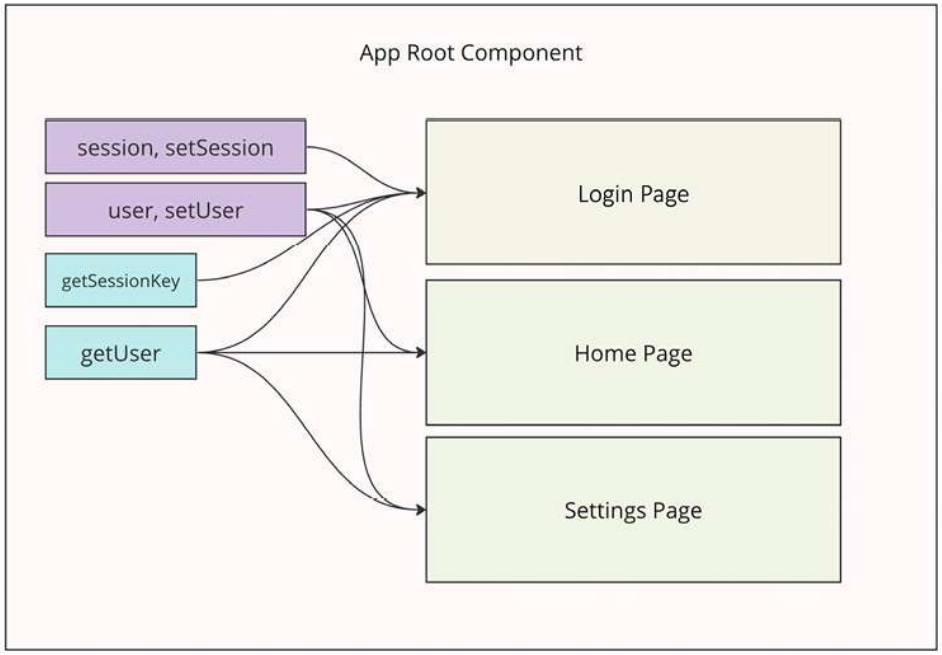

Figure 12.3: App root component

This image clearly demonstrates how the application becomes more complicated when we need to pass down all the necessary data and methods from the topmost component of the application to all its pages and components.

In addition to the complexity of implementing and maintaining such an approach to organizing the application's state, there is also a significant performance problem. For example, having a state in the root component created through `useState`, every time we update it, the entire application will be re-rendered because the app root component will be redrawn.

So, we have identified the main problems with organizing local state in the components of a large application:

- Overcomplicated component tree, where all important data must be passed down from top to bottom using props. This tightly couples the components, complicating the code and its maintenance.
- Performance issue, where the application may re-render unnecessarily when it's not required.

Looking at the last image, one can think of whether it is possible to break the connection of our components and extract all

the data and logic somewhere outside of the components. This is where the concept of global state comes into play.

Global state is a data management approach that allows state to be accessible and modifiable across different levels and components of your application. This solution overcomes the limitations of local state, facilitating data exchange between components and improving state manageability in large-scale projects.

To clearly understand how global state would look in our example, take a look at the image below:

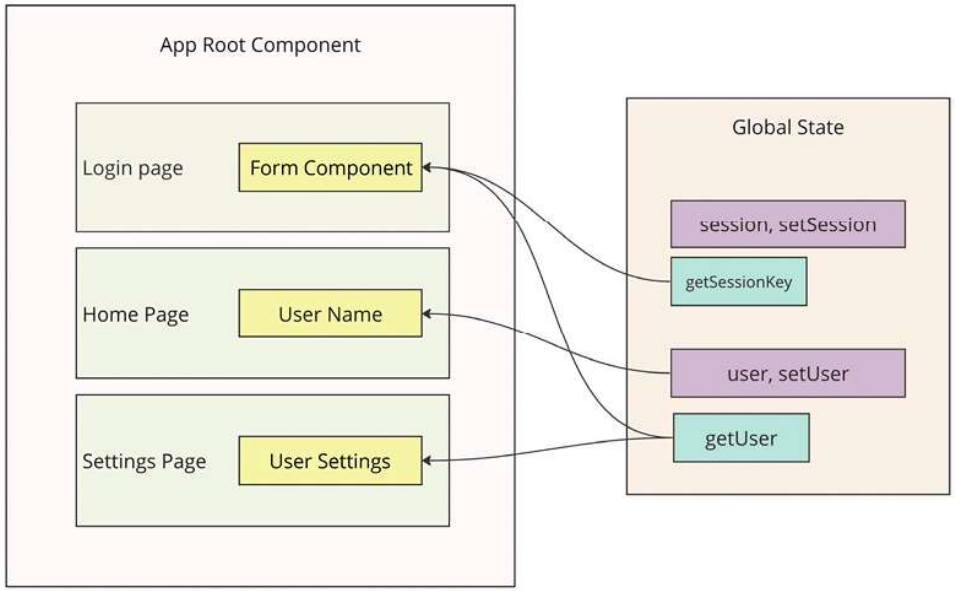

Figure 12.4: App root component and global state

In this example, we have a global state that is located outside of the components and the entire tree. Only the components that actually need any data from the state can directly access it and subscribe to its changes.

By implementing global state, we can solve both problems at once:

- Simplifies the component tree and dependencies, thereby scaling and supporting the application.
- Increases application performance because now, only those components that were subscribed to data from the global state are re-rendered when the state changes.

However, it's important to understand that the local state remains a very powerful tool and should not be abandoned in favor of the global state. We only gain advantages when the state needs to be used across different levels of application components. Otherwise, if we start transferring all variables and states to the global state, we will only complicate the application without gaining any benefits.

Now that we know that the global state is merely a way of organizing data, how do we manage the global state? A **state manager** is a tool that helps organize and manage state in an application, especially when it comes to complex interactions and extensive data. It provides a **centralized repository** for all

your application's state and manages its updates in an orderly and predictable manner. In practice, state managers are often represented as npm packages installed as project dependencies. However, it is also possible to manage the global state independently without any libraries using React's API. We will explore one such approach later on.

React Context API and useReducer

To organize the global state on your own, you can use tools that already exist in the React ecosystem, namely the **Context API** and `useReducer`. They represent a powerful duo for managing state, especially in situations where using third-party state managers seems excessive. These tools are ideal for creating and managing global states in more compact applications.

The **React Context API** is designed to pass data through the component tree without the need to pass props at every level. This simplifies access to data in deeply nested components and reduces **prop drilling** (passing props through many levels), as illustrated in *Figure 12.4*. The React Context API is particularly useful for data such as theme settings, language preferences, or user information.

Here's an example of how to store theme settings using context:

```
const ThemeContext = createContext();
const ThemeProvider = ({ children }) => {
  const theme = 'dark';
  return (
    <ThemeContext.Provider value={theme}>
      {children}
    </ThemeContext.Provider>
  );
};
const useTheme = () => useContext(ThemeContext);
export { ThemeProvider, useTheme };
```

In this example, we created `ThemeContext` using the `createContext` function. Then, we made a `ThemeProvider` component, which should wrap the root component of the application. This will later allow access at any level of nested components using the `useTheme` hook, which was created with the `useContext` hook:

```
const MyComponent = () => {
  const theme = useTheme();
  return (
    <div>
      <p>Current theme: {theme}</p>
    </div>
```

```
    );
  };
```

On any level of the component tree, we can access the current theme using the `useTheme` hook.

Next let's take a look at the next one of the duo, the special hook that will help us to build the global state. `useReducer` is a hook that allows you to manage complex states with reducers: functions that take the current state and an action, and then return a new state. `useReducer` is ideal for managing states that require complex logic or multiple sub-states. Let's consider a small example of a counter using `useReducer`:

```
import React, { useReducer } from 'react';
const initialState = { count: 0 };
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      throw new Error();
  }
}
function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);
```

```
    return (
      <>
        Count: {state.count}
        <button onClick={() => dispatch({ type: 'increment' })}>+</button>
        <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      </>
    );
  }
```

In this example, a reducer is implemented that has two actions: `increasing` and `decreasing` the counter.

The combination of the Context API and `useReducer` provides a powerful mechanism for creating and managing the global state of an application. This approach is convenient for small applications, where ready-made and larger state management solutions might be redundant. However, it's also worth noting that this solution doesn't completely solve the performance issue, as any change in the theme in the `useTheme` example or the counter in the counter example will cause the provider, and thus the entire component tree, to re-render. This can be avoided, but it requires additional logic and coding.

Therefore, more complex applications require a more powerful tool. For this, there are several ready-made and popular solutions for working with state, each with its unique features and suitable for different use cases.

Redux

The first of such tools is, of course, **Redux**. It is one of the most popular tools for managing state in complex JavaScript applications, especially when used with React. Redux provides predictable state management by maintaining the application's state in a single global object, simplifying the tracking of changes and data management.

Redux is based on three core principles: a single source of truth (one global state), the state is read-only (immutable), and changes are made using pure functions (reducers). These principles ensure an orderly and controlled data flow.

```
function counterReducer(state = { count: 0 }, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    default:
      return state;
  }
}
const store = createStore(counterReducer);
store.subscribe(() => console.log(store.getState()));
store.dispatch({ type: 'INCREMENT' });
store.dispatch({ type: 'DECREMENT' });
```

In this example, the state of the application has been implemented from the counter example. We have a `counterReducer`, which is a regular function that takes the current state and the action to be performed on it. The reducer always returns a new state.

Implementing asynchronous operations in the Redux world is a complex issue, as out of the box it offers nothing but middleware, which is used by third-party solutions. One such solution is `redux-thunk`.

`redux-thunk` is a middleware that allows you to call action creator functions that return a function instead of an action object. This provides the ability to delay the dispatch of an action or dispatch multiple actions by making asynchronous requests.

```
function fetchUserData() {
  return (dispatch) => {
    dispatch({ type: 'LOADING_USER_DATA' });
    fetch('/api/user')
      .then((response) => response.json())
      .then((data) => dispatch({ type: 'FETCH_USER_DATA_SUCCESS', payload: data }))
      .catch((error) => dispatch({ type: 'FETCH_USER_DATA_ERROR', error }));
  };
}
```

```
const store = createStore(reducer, applyMiddleware(thunk));
store.dispatch(fetchUserData());
```

As you can see in the example, we create a function, `fetchUserData`, that doesn't immediately change the state. Instead, it returns another function with a `dispatch` argument. This `dispatch` can be used as many times as needed to change the state.

There are also other more powerful but more complex solutions for asynchronous operations. We will not discuss these here.

Redux is well suited for managing complex global state in applications. It offers powerful debugging tools, such as time travel. Redux also facilitates the testing of state and logic due to the clear separation between data and its processing.

To integrate Redux with React, the `React-Redux` library is used. It provides `Provider` components, and the `useSelector` and `useDispatch` hooks, which allow easy connection of the Redux store to your React application.

```
function Counter() {
  const count = useSelector((state) => state.count);
  const dispatch = useDispatch();
  return (
```

```
      <div>
        <div>Count: {count}</div>
        <button onClick={() => dispatch({ type: 'INCREMENT' })}>+</button>
        <button onClick={() => dispatch({ type: 'DECREMENT' })}>-</button>
      </div>
    );
  }
```

In the example above, the `Counter` component works with the Redux state by subscribing to changes through `useSelector`. This subscription is more granular, and changing the counter does not lead to the re-rendering of the entire application, but only of the specific component that invokes this hook.

However, it's important to note the drawbacks of Redux. Although it is the most popular solution, it has significant issues that affect my personal choice against this solution:

- Redux is verbose. Implementing a large global state requires writing a lot of boilerplate code in the form of reducers, actions, selectors, etc.
- With the growth of the project, the complexity of maintaining and scaling the Redux state increases disproportionately.

As the project and global state grow, application performance significantly decreases. This happens due to the need for a

large number of computations, even if you simply change the state of one value from `false` to `true`.

Implementation of asynchronous operations is not supported out of the box by Redux and requires additional solutions, further complicating the understanding and maintenance of the project.

Dividing state and business logic into chunks for lazy loading requires a lot of effort. As a result, the application's size and therefore its initial loading speed are affected.

Despite these drawbacks, many companies and developers still use this solution, as it suits most business tasks, so I believe it is important to know this tool and be able to work with it.

MobX

The next popular solution for managing the global state is the **MobX** library. This library differs significantly from Redux, with a concept that is in some ways even the opposite.

MobX is a state management library that provides reactive and flexible interaction with data. Its main idea is to make the application state as simple and transparent as possible, working through small objects and classes that can be created as many times as desired and nested within each other.

Technically, the library allows for creating not just one global state but many small objects directly linked to some functionality of the application, which gives a significant advantage when working with large applications. To get the difference between one global state and MobX states, you can look at the following diagram:

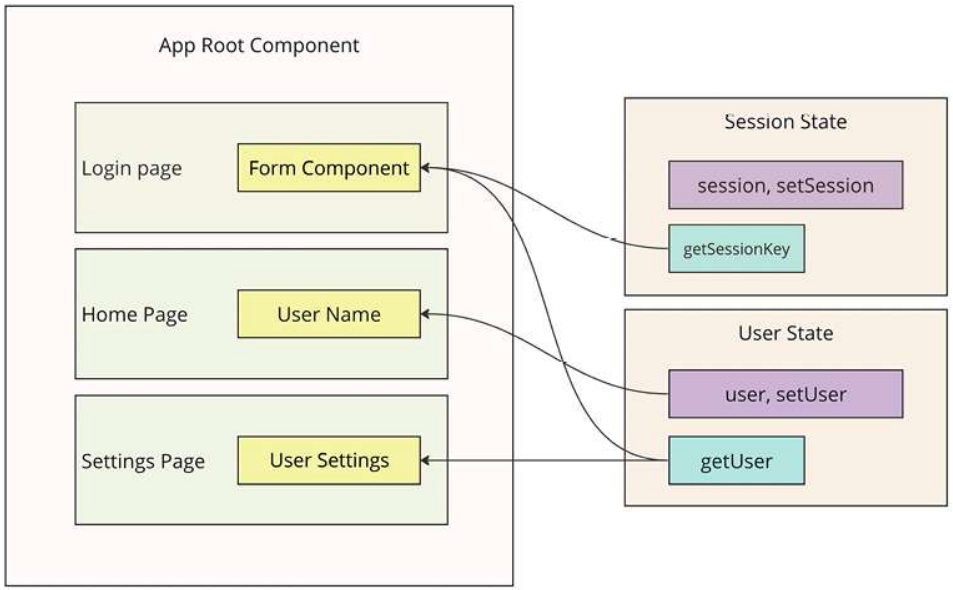

Figure 12.5: MobX state

In MobX, the state of the application is managed using `observable` method, which automatically track changes and inform related computed values and reactions. This allows the application to automatically update in response to state changes, simplifying the data flow and increasing flexibility.

```
class Store {
  @observable accessor count = 0;
  @computed get doubleCount() {
    return this.count * 2;
  }
  @action increment() {
    this.count += 1;
  }
  @action decrement() {
    this.count -= 1;
  }
}
const myStore = new Store();
```

In the example, the same counter is implemented using MobX. In one class, both the actual data and computed data are present, along with actions to change the state.

Speaking about asynchronous operations, MobX doesn't have any issues with that, as you can work in a regular class and add a new method that returns a promise.

```
class Store {
  @observable count = 0;
  @computed get doubleCount() {
    return this.count * 2;
  }
  @action increment() {
```

```
      this.count += 1;
    }
    @action decrement() {
      this.count -= 1;
    }
    @action async fetchCountFromServer() {
      const response = await fetch('/count');
      const data = await response.json();
      this.count = data.count;
    }
  }
  const myStore = new Store();
```

MobX is well suited for applications that require high performance and simplicity in managing complex data dependencies. It offers an elegant and intuitive way to handle complex state, allowing developers to focus on business logic rather than state management.

One drawback of this library is the considerable freedom it provides in organizing state, which can lead to difficulties and scalability issues in inexperienced hands. For example, MobX allows direct manipulation of object data, which can trigger component updates, but this can also lead to unexpected state changes in large projects and debugging challenges. Similarly, this freedom often results in small, clean MobX classes becoming tightly coupled, making testing and project development more challenging.

To integrate MobX with React, the `mobx-react` library is used, which provides the `observer` function. This allows React components to automatically react to changes in observed data.

```
import React from 'react';
import { observer } from 'mobx-react';
import myStore from './myStore';
const Counter = observer(() => {
  return (
    <div>
      <div>Count: {myStore.count}</div>
      <div>Double: {myStore.doubleCount}</div>
      <button onClick={() => myStore.increment()}>-</button>
      <button onClick={() => myStore.decrement()}>+</button>
    </div>
  );
});
```

In the example, the same counter is implemented using MobX. As you can see, we don't use hooks to access the state or providers to store it in the application context. We simply import the variable from the file and use it. The `myStore` created from the `Store` class is the state itself. It's easy to use the observed value of an object in a component because the component immediately subscribes to all changes of that value and will re-render every time it changes.

Just from the examples, you can already see how simple and convenient MobX is for managing state. Since it's just an object, there are no complexities in lazily loading it when needed and clearing the cache and memory of the application when the data is no longer needed. I consider it a powerful tool for state management and highly recommend trying it in a real project.

Summary

In this chapter, we've learned about global state and how to manage it. Using the example of limited local state, we've discussed why it's important to have global state in cases where shared data is needed across different components at different levels of the application.

We've explored an example using the React Context API and identified when to use it and when to prefer more powerful state management solutions. Next, we looked at two such solutions in the form of Redux and MobX.

In the next chapter, we will discuss server-side rendering and the benefits it can bring to our applications.